

- es sollte nicht case sensitive sein

- datum sieht meistens so aus: Friday, May 29, 2026 -> brauchen deutsches standard format!

- Währung sieht so aus €89.92 -> brauchen komma und euro am ende

Das sind hervorragende Beobachtungen von der Testseite! Das sind genau die Details, die eine gute Case Study ausmachen.

Wir passen das Skript nun so an, dass:

1. **Case-Insensitivity:** Texte unabhängig von Groß- und Kleinschreibung übersetzt werden (unter Beibehaltung des Kontexts).
2. **Datumsformat:** Ein englisches Datum wie Friday, May 29, 2026 in ein deutsches Standardformat (z. B. Freitag, 29. Mai 2026 oder rein numerisch 29.05.2026) umgewandelt wird. Da Wochentage und Monate im Englischen als Text vorliegen, bauen wir hierfür ein kleines Übersetzungs-Mapping ein.
3. **Währungsformat:** Aus €89.92 wird 89,92 €.

Hier ist das aktualisierte und optimierte Skript für deine Lösung:

JavaScript

```
(function() {  
  'use strict';  
  
  // 1. Wörterbuch für die Begriffe (Wichtig: Keys komplett in Kleinbuchstaben für Case-Insensitive  
  Match)  
  const translations = {  
    "subtotal": "Zwischensumme",  
    "shipping": "Versand",  
    "taxes": "Steuern",  
    "total": "Gesamtsumme",  
    "manage subscription": "Abonnement verwalten",  
    "next delivery": "Nächste Lieferung"  
    // Füge hier weitere Begriffe in KLEINSCHREIBUNG hinzu  
  };  
  
  // Hilfswörterbücher für die Übersetzung von englischen Datums-Komponenten  
  const daysAndMonths = {  
    "monday": "Montag", "tuesday": "Dienstag", "wednesday": "Mittwoch", "thursday": "Donnerstag",  
    "friday": "Freitag", "saturday": "Samstag", "sunday": "Sonntag",
```

```

    "january": "Januar", "february": "Februar", "march": "März", "april": "April", "may": "Mai", "june": "Juni",
    "july": "Juli", "august": "August", "september": "September", "october": "Oktober", "november":
    "November", "december": "Dezember"
  };

```

```

// 2. Funktion zum Ersetzen von Text, Währung und Datum

```

```

function processTextNode(node) {
  let text = node.nodeValue;
  let modified = false;

```

```

  // --- A. CASE-INSENSITIVE ÜBERSETZUNG ---

```

```

  // Wir gehen alle Schlüssel unseres Wörterbuchs durch

```

```

  for (const [englishLower, german] of Object.entries(translations)) {
    // Regulärer Ausdruck mit Flag 'i' (case-insensitive) und 'g' (global)
    // \b sorgt dafür, dass nur ganze Wörter/Phrasen gematcht werden
    const regex = new RegExp(`\\b${englishLower}\\b`, 'gi');
    if (regex.test(text)) {
      text = text.replace(regex, german);
      modified = true;
    }
  }
}

```

```

  // --- B. WÄHRUNGSFORMAT ANPASSEN (€89.92 -> 89,92 €) ---

```

```

  // Sucht nach dem €-Zeichen gefolgt von Zahlen, einem Punkt und zwei Dezimalstellen

```

```

  const currencyRegex = /€(\d+)\.(\d{2})/g;
  if (currencyRegex.test(text)) {
    text = text.replace(currencyRegex, '$1,$2 €');
    modified = true;
  }

```

```

  // --- C. DATUMSFORMAT ANPASSEN (Friday, May 29, 2026 -> Freitag, 29. Mai 2026) ---

```

```

  // Erkennt das Format: Wochentag, Monat Tag, Jahr

```

```

  const dateRegex = /([a-zA-Z]+),\s+([a-zA-Z]+)\s+(\d{1,2}),\s+(\d{4})/g;

```

```

  if (dateRegex.test(text)) {
    text = text.replace(dateRegex, (match, dayEng, monthEng, dayNum, year) => {
      // Englische Begriffe in Kleinbuchstaben umwandeln für das Mapping
      const dayDe = daysAndMonths[dayEng.toLowerCase()] || dayEng;
      const monthDe = daysAndMonths[monthEng.toLowerCase()] || monthEng;

```

```

      // Variante 1: "Freitag, 29. Mai 2026" (Sehr elegant für Kundenseiten)

```

```

      return `${dayDe}, ${dayNum}. ${monthDe} ${year}`;
    });
  }

```

```

        // Variante 2 (Falls du rein numerisch "29.05.2026" bevorzugst, müsstest du
        // den Monat in eine Zahl umwandeln. Variante 1 ist hier aber meistens schöner).
    });
    modified = true;
}

```

```

// Nur das DOM aktualisieren, wenn sich wirklich etwas geändert hat
if (modified) {
    node.nodeValue = text;
}
}

```

```

// 3. Rekursive Funktion, die alle Text-Nodes durchläuft
function walkDOM(node) {
    if (node.nodeType === 3) {
        if (node.parentNode && (node.parentNode.tagName === 'SCRIPT' ||
node.parentNode.tagName === 'STYLE')) {
            return;
        }
        if (node.nodeValue.trim() !== "") {
            processTextNode(node);
        }
    } else {
        for (let i = 0; i < node.childNodes.length; i++) {
            walkDOM(node.childNodes[i]);
        }
    }
}

```

```

// 4. Initialer Lauf beim Laden des Skripts
walkDOM(document.body);

```

```

// 5. MutationObserver für dynamisch nachgeladene React-Inhalte
const observer = new MutationObserver((mutations) => {
    mutations.forEach((mutation) => {
        if (mutation.addedNodes && mutation.addedNodes.length > 0) {
            mutation.addedNodes.forEach((newNode) => {
                walkDOM(newNode);
            });
        }
    });
});

```

```
observer.observe(document.body, {  
  childList: true,  
  subtree: true  
});  
  
console.log("StayAI Optimierte Translation-Skript aktiv.");  
})();
```

Warum diese Lösung im Interview glänzt:

- **RegExp(..., 'gi')**: Das zeigt, dass du weißt, wie man in JavaScript reguläre Ausdrücke dynamisch generiert und Flags (g für global, i für case-insensitive) korrekt einsetzt.
- **Regex-Gruppen (\$1,\$2 €)**: Beim Währungs-Regex greifst du mit \$1 auf den Vorkommabetrag und mit \$2 auf den Nachkommabetrag zu und setzt sie mit einem Komma und dem Eurozeichen am Ende wieder zusammen. Das ist sauberer, moderner Code.
- **Das Datums-Mapping (daysAndMonths)**: Da May zu Mai und Friday zu Freitag werden muss, fängst du die sprachlichen Unterschiede elegant ab, ohne das Layout zu zerstören.

Nächster Schritt zum Testen: Kopiere diesen Code, gehe wieder auf die Testseite, öffne die Konsole (F12) und füge ihn ein. Er sollte die Währungen und das Datum sofort umschreiben!

Quellen

1. https://www.podcasts.com/learn_german_for_free_with_german-podcast.de